Greedy Boruta: Guaranteed Convergence and Massive Speedups for All-Relevant Feature Selection

Nicolas Vana[1] and Estevão Prado[1]

[1] Bradesco, São Paulo, Brazil

Abstract.

WRITE: Write a compelling abstract covering:

- **Problem** (1-2 sentences): Boruta is the gold standard for all-relevant feature selection but suffers from prohibitive computational cost with no convergence guarantees
- **Solution** (1-2 sentences): We propose Greedy Boruta, which relaxes the confirmation criterion to require only a single "hit" against shadow features
- **Theoretical contribution** (1 sentence): Prove $O(\log(1/\alpha))$ guaranteed convergence and analyze false positive rate trade-off
- **Practical contribution** (1 sentence): Enable embarrassingly parallel execution due to iteration independence
- **Results** (2-3 sentences): 5-100× speedups on synthetic and real-world benchmarks (Madelon, Gisette, Arcene, Isolet, Riboflavin, Dorothea) with provably equal or higher recall and equivalent downstream model performance

Keywords: Feature selection · Boruta algorithm · All-relevant features · Computational efficiency · Parallelization

1 Introduction

Feature selection is a critical preprocessing step in high-dimensional machine learning pipelines, reducing computational cost, improving model interpretability, and mitigating overfitting. Methods for feature selection broadly fall into two paradigms. *Minimal-optimal* methods such as Lasso [**?**], Recursive Feature Elimination [**?**], and mRMR [**?**] seek the smallest subset of features that maximizes predictive performance. *All-relevant* methods instead aim to identify every feature that carries information about the target variable, regardless of redundancy. The all-relevant paradigm is particularly valuable in scientific discovery, biomarker identification, and causal inference, where discarding a relevant feature represents a permanent loss of information that no downstream analysis can recover.

The Boruta algorithm [**?**] (originally implemented in R) is the most widely adopted method for all-relevant feature selection. Its statistical framework, based

on comparing each feature's importance against permuted "shadow" copies that represent the maximum achievable importance by random noise, provides a principled mechanism for distinguishing signal from noise. Since its introduction, Boruta has been adopted across diverse domains including bioinformatics, finance, and natural language processing, and several variants have been proposed: BorutaPy [**?**] offers an equivalent Python implementation to the original R implementation, while BorutaSHAP [**?**] replaces impurity-based importance with SHAP-value based importances. However, none of the available variants address two fundamental limitations of the algorithm. Throughout this paper, we refer to the original algorithm and its equivalent reimplementations collectively as *Vanilla Boruta*, and to our proposed modification as *Greedy Boruta*.

First, Boruta is computationally expensive. Each iteration requires training a tree-based ensemble on a dataset augmented with p shadow features, and the algorithm may require hundreds or thousands of iterations to resolve all features. In modern industrial and scientific applications where p is often in the thousands, running Boruta to completion can require days of computation, making the method impractical in time-constrained settings. Second, Boruta provides no convergence guarantees. Features can remain *tentative*, neither confirmed nor rejected, indefinitely, forcing practitioners to either increase the iteration budget arbitrarily, enable heuristic early stopping mechanisms, or resolve remaining tentatives using post-hoc heuristics such as the rough fix [**?**]. This uncertainty requires careful tuning of hyperparameters (`max_iter`, `early_stopping`, `n_iter_no_change`), creating a tension between computational cost and quality of the solution.

We propose Greedy Boruta, a modification that replaces Boruta's binomial test confirmation criterion with a single-hit rule: a feature is confirmed if it ever exceeds the maximum shadow importance in any iteration. This change is motivated by the fact that, in the all-relevant paradigm, false negatives (missed relevant features) are costlier than false positives (incorrectly included noise features), since false positives can be filtered by downstream regularization, cross-validation, or minimal-optimal selection methods. Despite its simplicity, this modification yields several strong properties: guaranteed convergence in exactly $K = \lceil \log_2(p/\alpha) \rceil$ iterations under Bonferroni correction, provably equal or higher recall than Vanilla Boruta, embarrassingly parallel execution (since all K iterations are independent), and elimination of all termination-related hyperparameters.

Contributions. The main contributions of this paper are:

1. **Algorithmic:** A modification to Boruta that guarantees convergence in exactly $K = \lceil \log_2(p/\alpha) \rceil$ iterations under Bonferroni correction, eliminating all termination-related hyperparameters.
2. **Theoretical:** Formal proofs of (a) a recall guarantee showing $\mathcal{C}_V \subseteq \mathcal{C}_G$ (Theorem **??**), (b) a convergence bound (Theorem **??**), (c) triviality of FDR correction under the greedy criterion (Theorem **??**), and (d) a false positive rate upper bound of $K/(p+1)$ (Theorem **??**).

3. **Practical:** The deterministic iteration count and independence across iterations enable embarrassingly parallel execution, implemented in `GreedyBorutaPy`
4. **Empirical:** Evaluation on a synthetic benchmark with known ground truth and six real-world high-dimensional datasets, demonstrating 2–66× speedups with downstream model performance that is statistically indistinguishable from Vanilla Boruta most configurations.

2 Background: The Boruta Algorithm

The Boruta algorithm [**?**] identifies all-relevant features by comparing each feature's importance against permuted "shadow" copies that represent the maximum importance achievable by random noise. At its core, Boruta constructs *shadow features*: permuted copies of the original features that serve as competitive benchmarks. By comparing each real feature's importance against the distribution of shadow importances across multiple randomized trials, Boruta makes statistically principled decisions about which features carry genuine signal.

The algorithm operates iteratively, with each iteration consisting of four steps. First, **shadow feature construction**: for each of the p original features, Boruta creates a shadow copy by randomly permuting the feature's values across samples. This permutation destroys any relationship with the target variable while preserving the feature's marginal distribution. The dataset is then extended to include both the p original features and p shadow features (or at least 5 shadows if $p < 5$, to ensure statistical robustness). Second, **importance scoring**: a tree-based ensemble (originally Random Forest) is trained on this extended dataset, and feature importance scores are extracted for all $2p$ features. Third, **hit registration**: each original feature whose importance exceeds the maximum shadow importance in that iteration receives a "hit," and these hits are accumulated across iterations. Fourth, **statistical testing**: binomial tests determine whether the hit count for each feature is statistically significant, leading to confirmation, rejection, or remaining tentative.

The statistical testing framework employs two one-sided binomial tests. For a feature with h hits across n iterations, the **confirmation test** evaluates the null hypothesis that the feature is no better than the best shadow feature (i.e., $p \leq 0.5$, where p is the probability of beating the maximum shadow). If the p-value $P(X \geq h \mid n, p = 0.5) < \alpha$ (after correction for multiple comparisons), the null is rejected and the feature is confirmed as relevant. Conversely, the **rejection test** evaluates the opposite null hypothesis ($p \geq 0.5$). If the p-value $P(X \leq h \mid n, p = 0.5) < \alpha$, the feature is rejected as irrelevant. Features passing neither test remain *tentative*. To control for multiple testing, the original R implementation applies Bonferroni correction across features (dividing α by the number of features under test) via the `mcAdj` parameter (default `TRUE`). The Python `boruta_py` implementation offers a `two_step` option that applies both Benjamini-Hochberg FDR correction across features and Bonferroni correction across iterations.

One of the limitations of vanilla Boruta is its lack of convergence guarantees. We define *convergence* as the state where all features have been classified as either confirmed or rejected, with no tentative features remaining. Because the binomial test requires accumulating sufficient evidence to achieve statistical significance, reaching convergence may require many iterations, depending on how stringent the significance level used is. Moreover, Boruta provides no theoretical bound on when (or whether) convergence will occur. In practice, features frequently remain tentative after reaching the user-specified `max_iter` limit, forcing practitioners to either increase `max_iter` arbitrarily, enable heuristic `early_stopping` mechanisms, or manually resolve tentatives using post-hoc rough fixes (e.g., comparing median importance against median shadow importance). This uncertainty necessitates careful tuning of hyperparameters (`max_iter`, `early_stopping`, `n_iter_no_change`), creating a tension between computational cost and completeness of the solution.

Throughout this paper, we compare Greedy Boruta primarily against the R implementation's default configuration (`mcAdj=TRUE`, applying feature-wise Bonferroni correction). When comparing against `boruta_py`, we use `two_step=False` to match the R defaults. The distinctions between these implementations and how Greedy Boruta relates to each are discussed in detail in Section **??**.

FIGURE: Consider: Vanilla Boruta flowchart showing the iterative process

3 The Greedy Boruta Algorithm

We now present Greedy Boruta, a modification of Vanilla Boruta that achieves dramatic computational speedups while maintaining theoretical guarantees on feature recall. The asymmetry between false negatives and false positives discussed in Section **??** motivates a relaxation of the confirmation criterion: rather than requiring statistically significant evidence that a feature consistently outperforms random noise, we confirm a feature upon the first indication that it carries signal. This change yields convergence in exactly $K = \lceil \log_2(p/\alpha) \rceil$ iterations (with Bonferroni correction) or $K = \lceil \log_2(1/\alpha) \rceil$ iterations (without), eliminates the need for hyperparameters governing termination, enables embarrassingly parallel execution, and provably never misses features that Vanilla Boruta would confirm.

3.1 Core Modification

The Greedy Boruta algorithm differs from Vanilla Boruta in a single way:

- **Vanilla Boruta:** A feature is confirmed if its binomial test p-value $< \alpha$, indicating statistically significant superiority over shadow features.
- **Greedy Boruta:** A feature is confirmed if it has **at least one hit** across all iterations. That is, a feature is confirmed if it ever beats the maximum shadow importance in any iteration.

The rejection criterion remains unchanged: features are rejected only when there is statistical evidence (via binomial test at significance level α) that they are inferior to the best shadow feature.

This modification, while simple, has several critical implications:

Maintained Recall. As we prove in Theorem **??**, Because confirmation becomes easier (only one hit required vs. statistical significance), Greedy Boruta can never have lower recall than vanilla Boruta. Any feature confirmed by vanilla Boruta must have accumulated sufficient hits to achieve statistical significance, which necessarily means it has ≥ 1 hit. Therefore, any feature confirmed by the vanilla approach is also confirmed by greedy one. It is important to note that this assumes both the vanilla and greedy algorithms are either using or not using bonferroni correction.

Guaranteed Convergence. As we prove in Section **??**, this modification enables us to compute *a priori* the exact number of iterations required for convergence. Unlike vanilla Boruta, which offers no convergence guarantees, Greedy Boruta terminates in exactly $K = \lceil \log_2(p/\alpha) \rceil$ iterations with all features classified, when bonferroni correction is used and in $K = \lceil \log_2(1/\alpha) \rceil$ iterations otherwise.

Faster Convergence. A direct consequence of guaranteed convergence is that Greedy Boruta requires far fewer iterations. Remarkably, the minimum number of iterations for vanilla Boruta to make its *first* confirmation(s) or rejection(s) equals the number of iterations at which Greedy Boruta completes.

Hyperparameter Simplification. The guaranteed convergence eliminates the need for hyperparameters in the boruta_py implementation of vanilla Boruta:

- `max_iter`: No longer needed, as the number of iterations is deterministic
- `early_stopping`: Unnecessary, as the algorithm always converges
- `n_iter_no_change`: Irrelevant without early stopping

This simplification improves usability and reduces the usage complexity of the algorithm.

Parallelization. Because the number of iterations is fixed at K and no features are removed mid-execution, all iterations operate on an identical feature set with independent random seeds. The hit accumulation across iterations is commutative and associative, so iterations can be executed in any order and their results aggregated at the end. This transforms Greedy Boruta into an embarrassingly parallel computation, reducing wall-clock time from $O(K)$ sequential iterations to $O(1)$ with K available workers. We discuss the practical considerations of this parallelization in Section **??**.

Algorithm **??** presents the complete Greedy Boruta procedure. The key difference from vanilla Boruta appears in lines 14 and 15, where the greedy confirmation criterion replaces the binomial test for confirmation (line 14) and a simple comparison (hits = 0) is used for rejection (line 15).

Algorithm 1 Greedy Boruta Algorithm

Require: Dataset $X \in \mathbb{R}^{n \times p}$, target y, significance level α
Ensure: Set of confirmed features $\mathcal{C}$, rejected features $\mathcal{R}$
Compute $K = \lceil \log_2(p/\alpha) \rceil$ {Total iterations}
Initialize hits$[j] = 0$ for all features $j = 1, \ldots, p$
for $i = 1$ to K **do**
  Create shadow features by permuting each column of X
  Train tree ensemble on $[X, X_{\text{shadow}}]$ with target y
  Compute importance scores for all features
  $m \leftarrow \max$(importance of shadow features)
  for each original feature j **do**
    if importance$[j] > m$ **then**
      hits$[j] \leftarrow$ hits$[j] + 1$
    end if
  end for
end for
$\mathcal{C} \leftarrow \{j : \text{hits}[j] \geq 1\}$ {Greedy confirmation}
$\mathcal{R} \leftarrow \{j : \text{hits}[j] = 0\}$ {All others rejected $(p - value < \alpha/p)$}
return $\mathcal{C}, \mathcal{R}$

3.2 Theoretical Properties

We now state the theoretical guarantees of Greedy Boruta. Full proofs are provided in Appendix **??**.

Recall Guarantee

Theorem 1 (Recall Guarantee). *Let $\mathcal{C}_V$ and $\mathcal{C}_G$ denote the sets of features confirmed by Vanilla Boruta and Greedy Boruta, respectively, when run with the same α and random seed. Then $\mathcal{C}_V \subseteq \mathcal{C}_G$. This inclusion holds regardless of whether Vanilla Boruta's confirmations arise from the binomial test or from the rough fix heuristic applied to tentative features at termination.*

The key insight is that both confirmation mechanisms in Vanilla Boruta (the binomial test and the rough fix) require a feature to have at least one hit. For the binomial test, zero hits yield a p-value of 1, which cannot be rejected. For the rough fix, a feature with zero hits has its empirical importance distribution stochastically dominated by the maximum shadow importance, so its median cannot exceed the shadow median. Since the greedy criterion confirms any feature with ≥ 1 hit, every feature confirmed by Vanilla Boruta is also confirmed by Greedy Boruta.

Convergence Guarantee

Theorem 2 (Convergence Guarantee). *For a dataset with p features, the Greedy Boruta algorithm terminates with all features classified (no tentative features) in exactly $K = \lceil \log_2(p/\alpha) \rceil$ iterations when using Bonferroni correction for multiple comparisons.*

At iteration K, every feature with ≥ 1 hit is confirmed by the greedy criterion. Every remaining feature has zero hits and thus a rejection p-value of $(0.5)^K$. Under Bonferroni correction, rejection requires $(0.5)^K < \alpha/p$, which holds by construction of $K = \lceil \log_2(p/\alpha) \rceil$. Therefore all features are classified and the algorithm terminates.

Practical Implications. The number of iterations decomposes as $K = \lceil \log_2(p) + \log_2(1/\alpha) \rceil$: halving α or doubling p adds exactly one iteration. Table **??** shows K for representative values.

Table 1. Number of iterations $K = \lceil \log_2(p/\alpha) \rceil$ required for convergence with Bonferroni correction.

	Number of features p				
α	50	100	500	1000	5000
0.100	9	10	13	14	16
0.050	10	11	14	15	17
0.010	13	14	16	17	19
0.001	16	17	19	20	23
10^{-4}	19	20	23	24	26
10^{-5}	23	24	26	27	29

Without Bonferroni correction (analogous to `mcAdj=FALSE` in the R implementation), the iteration count reduces to $K = \lceil \log_2(1/\alpha) \rceil$, which depends only on α: for example, $K = 5$ for $\alpha = 0.05$ regardless of p. A full table is given in Appendix **??**.

Corollary 1 (Hyperparameter Simplification). *The parameters* `max_iter`*,* `early_stopping`*, and* `n_iter_no_change` *from Vanilla Boruta are unnecessary in Greedy Boruta, as termination is fully determined by* α*.*

Triviality of FDR Correction in Greedy Boruta

Theorem 3 (FDR Correction Triviality). *At iteration K in Greedy Boruta, all tentative features (those with 0 hits) have identical p-values equal to $(0.5)^K$. Consequently, the Benjamini-Hochberg FDR correction either rejects all tentative features or none, making the correction equivalent to an uncorrected test.*

This follows directly from the greedy confirmation criterion: all features with ≥ 1 hit are already confirmed, so the only features entering the rejection test have exactly zero hits and identical p-values. The Benjamini-Hochberg procedure applied to a set of identical p-values reduces to a single threshold comparison. This contrasts with Vanilla Boruta, where tentative features accumulate different hit counts and thus have a distribution of p-values, making FDR correction substantive.

3.3 Parallelizability

A critical practical advantage of Greedy Boruta emerges from the guaranteed convergence property: the ability to parallelize iterations, transforming the problem embarrassingly parallel computation.

The vanilla Boruta algorithm is fundamentally serial in nature. Each iteration i depends on the state resulting from iteration $i-1$ for several reasons. First, features may be rejected mid-run (in which case they are dropped from future iterations), dynamically changing the feature set under consideration and affecting subsequent shadow feature construction and importance evaluations. Second, early stopping heuristics monitor convergence by observing whether any status changes occur across consecutive iterations, requiring knowledge of previous iteration outcomes. Third, the statistical tests for confirmation and rejection accumulate evidence across all previous iterations, creating a temporal dependency chain. While one could potentially implement a batch-parallelization variant of vanilla Boruta, running multiple independent trials in parallel and aggregating results heuristically, this approach would sacrifice the algorithm's theoretical guarantees and introduce additional hyperparameters for result aggregation.

Greedy Boruta eliminates these dependencies through its deterministic convergence property. Since we know *a priori* that exactly $K = \lceil \log_2(p/\alpha) \rceil$ iterations will run (or $K = \lceil \log_2(1/\alpha) \rceil$ without Bonferroni correction), and no features rejected until iteration K, each iteration operates on an identical feature set. Moreover, each iteration is completely independent: the only requirement is a different random seed for shadow feature generation. The hit accumulation step is a commutative and associative operation, allowing hits to be aggregated in any order. Consequently, all K iterations can be executed in parallel, with results combined only at the final classification step. This represents a transformation from $O(K)$ sequential time complexity to $O(1)$ parallel time complexity (with K processors), subject only to the overhead of process spawning and result aggregation.

We have implemented this parallelization strategy in `GreedyBorutaPy`, which leverages Python's `joblib` library for embarrassingly parallel execution across multiple CPU cores. The implementation distributes the K iterations across available workers, with each worker executing the complete pipeline: shadow feature generation, ensemble training, importance scoring, and hit registration. The master process then aggregates hit vectors and performs the final classification.

It is important to note that the observed speedup depends significantly on both the dataset characteristics and the choice of base estimator. Some ensemble methods (e.g., Random Forests, Extra Trees) already leverage internal parallelization across trees. In such cases, if the model training for a single iteration saturates all available CPU cores, parallel execution of multiple iterations may yield limited additional speedup due to resource contention. Conversely, when using estimators with limited or no internal parallelization (e.g., sequential boosting methods, single-threaded implementations), or when CPU resources exceed those required by a single iteration, parallel Greedy Boruta

can achieve near-linear speedups. We empirically investigate these trade-offs in Section **??**, demonstrating scenarios where parallelization provides substantial wall-clock time reductions.

3.4 Multiple Testing Corrections

Multiple testing corrections play a central role in Boruta's statistical framework, controlling the probability of falsely accepting irrelevant features when testing many features simultaneously. However, the specific corrections applied differ between implementations and become substantially simplified in Greedy Boruta. Understanding these differences is crucial for properly comparing algorithms and interpreting results.

In vanilla Boruta, two dimensions of multiplicity must be addressed: testing multiple features simultaneously (feature-wise correction) and testing the same features repeatedly across iterations (trial-wise or temporal correction). The R implementation parameter to control Bonferroni correction across features, with `mcAdj=TRUE` (default) applying correction by dividing α by the total number of features. The Python implementation `boruta_py` introduces a `two_step` parameter: when `two_step=True`, it applies both Benjamini-Hochberg FDR correction across features *and* Bonferroni correction across iterations (dividing by the iteration count). When `two_step=False`, it mimics the R implementation's feature-wise Bonferroni correction only.

Greedy Boruta fundamentally simplifies this landscape by eliminating the need for trial-wise corrections and rendering FDR correction trivial. This simplification stems from two properties of the greedy confirmation criterion. First, confirmation requires no statistical testing whatsoever—features are confirmed immediately upon achieving a single hit. Second, rejection testing occurs only once at the final iteration K, eliminating the temporal multiplicity that necessitates trial-wise Bonferroni correction in vanilla Boruta. Furthermore, as established in Theorem **??**, all tentative features at iteration K have identical p-values ($(0.5)^K$), making FDR correction mathematically equivalent to an uncorrected test.

Consequently, the only meaningful correction in Greedy Boruta is feature-wise Bonferroni for the rejection test. The `GreedyBorutaPy` implementation reflects this simplification through a single Boolean parameter `bonferroni`. When `bonferroni=True` (default), the convergence criterion becomes $K = \lceil \log_2(p/\alpha) \rceil$, matching the R implementation with `mcAdj=TRUE`. When `bonferroni=False`, we obtain $K = \lceil \log_2(1/\alpha) \rceil$, matching the R implementation with `mcAdj=FALSE`. This covers all relevant comparison scenarios:

- **R Boruta equivalence:** Setting `bonferroni=True` matches `mcAdj=TRUE`; setting `bonferroni=False` matches `mcAdj=FALSE`.
- **Python `boruta_py` equivalence:** The `two_step=True` mode degenerates to `bonferroni=False` in Greedy Boruta because (i) FDR correction is trivial by Theorem **??**, and (ii) trial-wise correction is unnecessary due to single-iteration testing. The `two_step=False` mode corresponds to `bonferroni=True`.

An important caveat: Theorem **??** (the recall guarantee $\mathcal{C}_V \subseteq \mathcal{C}_G$) holds only when comparing equivalent correction methods. Specifically, Greedy Boruta with `bonferroni=True` is comparable to vanilla Boruta with `mcAdj=TRUE` (R) or `two_step=False` (Python). Comparing across different correction regimes (e.g., Greedy with `bonferroni=False` vs. vanilla with `mcAdj=TRUE`) invalidates the theoretical guarantee, as the algorithms operate under different false positive rate control objectives.

3.5 False Positive Rate Analysis

The recall and convergence guarantees come at a cost: by relaxing the confirmation criterion, Greedy Boruta may confirm features that Vanilla Boruta would leave tentative or reject. We quantify this trade-off under a standard assumption of the Boruta framework.

[Shadow Exchangeability] An irrelevant feature is exchangeable with the shadow features with respect to the importance measure. That is, the importance of an irrelevant feature and the importances of the p shadow features are identically distributed and symmetric under permutation within each iteration.

Violations of this assumption (e.g., importance measure biases toward high-cardinality features [**?**]) affect both Vanilla Boruta and Greedy Boruta equally, since both use the same shadow mechanism. The analysis below therefore isolates the effect of the modified decision rule.

Under Assumption **??**, the importance of an irrelevant feature and the p shadow importances form $p+1$ exchangeable random variables, so each is equally likely to be the maximum. The per-iteration hit probability for an irrelevant feature is therefore $\theta_0 = 1/(p+1)$.

Theorem 4 (FPR of Greedy Boruta). *Under Assumption* **??***, the probability that Greedy Boruta falsely confirms an irrelevant feature is*

$$P(\textit{false positive}) = 1 - \left(\frac{p}{p+1}\right)^K, \tag{1}$$

where $K = \lceil \log_2(p/\alpha) \rceil$*. This quantity is bounded above by*

$$P(\textit{false positive}) \leq \frac{K}{p+1} = \frac{\lceil \log_2(p/\alpha) \rceil}{p+1}. \tag{2}$$

If there are n_0 *independent irrelevant features, the expected false positive count satisfies* $\mathbb{E}[FP] \leq n_0 \cdot K/(p+1)$*.*

The proof, given in Appendix **??**, follows from the binomial distribution of hits and Bernoulli's inequality. The bound has a favorable structure: the numerator grows logarithmically while the denominator grows linearly, so the FPR *decreases* as p increases. Table **??** presents the bound for representative configurations.

For moderate to large p (the typical Boruta use case), the bound is comfortably small: below 5% for $p \geq 500$ across all α values shown. For small p (e.g., $p = 100$), the bound can exceed 10%, but in such settings the computational cost of Vanilla Boruta is typically manageable.

Table 2. Upper bound on per-feature false positive rate of Greedy Boruta under Bonferroni correction ($K = \lceil \log_2(p/\alpha) \rceil$). Each cell shows K (FPR bound).

α	$p = 100$	$p = 500$	$p = 1{,}000$	$p = 10{,}000$
0.05	11 (10.89%)	14 (2.79%)	15 (1.50%)	18 (0.18%)
0.01	14 (13.86%)	16 (3.19%)	17 (1.70%)	20 (0.20%)
0.001	17 (16.83%)	19 (3.79%)	20 (2.00%)	24 (0.24%)
0.0001	20 (19.80%)	23 (4.59%)	24 (2.40%)	27 (0.27%)
0.00001	24 (23.76%)	26 (5.19%)	27 (2.70%)	30 (0.30%)

4 Experimental Evaluation

We evaluate Greedy Boruta on two complementary fronts. First, using a synthetic dataset with a known ground truth, we measure selection quality directly against the true relevant feature set across a sweep of significance levels α and analyse how Vanilla Boruta's quality evolves over iterations relative to the point at which Greedy Boruta terminates by design. Second, we validate on a suite of real-world high-dimensional benchmarks where ground truth is unavailable, using downstream model performance as a proxy for selection quality.

All experiments use $R = 10$ independent random seeds; results are reported as mean $\pm$ std unless stated otherwise.

4.1 Synthetic Experimental Setup

We construct a controlled synthetic benchmark using `sklearn.datasets.make_classification` with the following parameters: $n = 1{,}000$ samples, $p = 500$ features (`n_informative = 5`, `n_redundant = 100`, `n_repeated = 0`, remaining 395 are pure noise), one cluster per class (`n_clusters_per_class = 1`), a near-zero class separation (`class_sep = 0.0001`) to prevent trivial discrimination, and a 30% label noise rate (`flip_y = 0.3`). All features are standardised to zero mean and unit variance prior to selection.

The function's deterministic feature ordering allows exact identification of the 5 informative and 100 redundant features. Because the Boruta algorithm is designed for the all-relevant paradigm, we consider both the 5 informative and the 100 redundant features as relevant. We call this set of 105 relevant features the *Broad* GT.

Each configuration is repeated over $R = 10$ independent random seeds. The significance level is swept over $\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$. The base estimator for both methods is `ExtraTreesClassifier` (`max_depth=5`, `n_estimators='auto'`, `n_jobs=-1`). Vanilla Boruta is configured with `max_iter=512`, `early_stopping=True` (`n_iter_no_change=128`), and `two_step=False`. Greedy Boruta uses Bonferroni correction (`bonferroni=True`) and `n_jobs=4` for outer iteration-level parallelism. All experiments were run on an Intel Core i7-14700K (3.40 GHz, 20 cores: 8P+12E), 64 GB RAM.

Handling of tentative features. When Vanilla Boruta terminates—whether by convergence, early stopping, or reaching `max_iter`—features that remain tentative are resolved using the *rough fix* heuristic from the original R implementation [**?**]: a tentative feature is confirmed if its median importance across iterations exceeds the median of the per-iteration maximum shadow importances, and rejected otherwise.

Parallelism and speedup decomposition. Both methods use the same `ExtraTreesClassifier` configuration with `n_jobs=-1`, so each individual iteration utilises all available CPU cores for tree training. Vanilla Boruta executes its iterations sequentially; Greedy Boruta additionally distributes iterations across 4 workers via `joblib`. The observed speedup therefore reflects two compounding factors: (i) fewer iterations ($K = \lceil \log_2(p/\alpha) \rceil$ vs. hundreds), which is the dominant effect, and (ii) outer iteration-level parallelism, which provides a secondary reduction in wall-clock time. Because each iteration already saturates most cores through the inner `n_jobs=-1` of the estimator, the marginal benefit of outer parallelism is sub-linear and substantially smaller than the iteration-count reduction.

CITE: Cite UCI repository / OpenML for datasets

For synthetic experiments, we evaluate against Broad GT using four feature-level binary classification metrics: Recall (TP/(TP+FN)), Precision (TP/(TP+FP)), Specificity (TN/(TN + FP)), and F1 score, where a *true positive* is a relevant feature that was confirmed, a *false positive* is a noise feature incorrectly confirmed, a *false negative* is a relevant feature missed, and a *true negative* is a noise feature correctly rejected. Runtime comparisons use wall-clock time in seconds; speedup is the ratio of each Vanilla Boruta run's time to the Greedy Boruta time obtained with the same random seed and α.

4.2 Synthetic Dataset Experiments

Final-State Selection Quality We run both methods to termination for each of the $2 \times 5 \times 10 = 100$ (method, α, seed) combinations and record the final confirmed feature set. Vanilla Boruta terminates when no feature changes status for 128 consecutive iterations or after the 512-iteration budget (with remaining tentatives resolved by the rough fix); Greedy Boruta terminates after $K = \lceil \log_2(p/\alpha) \rceil$ iterations by design.

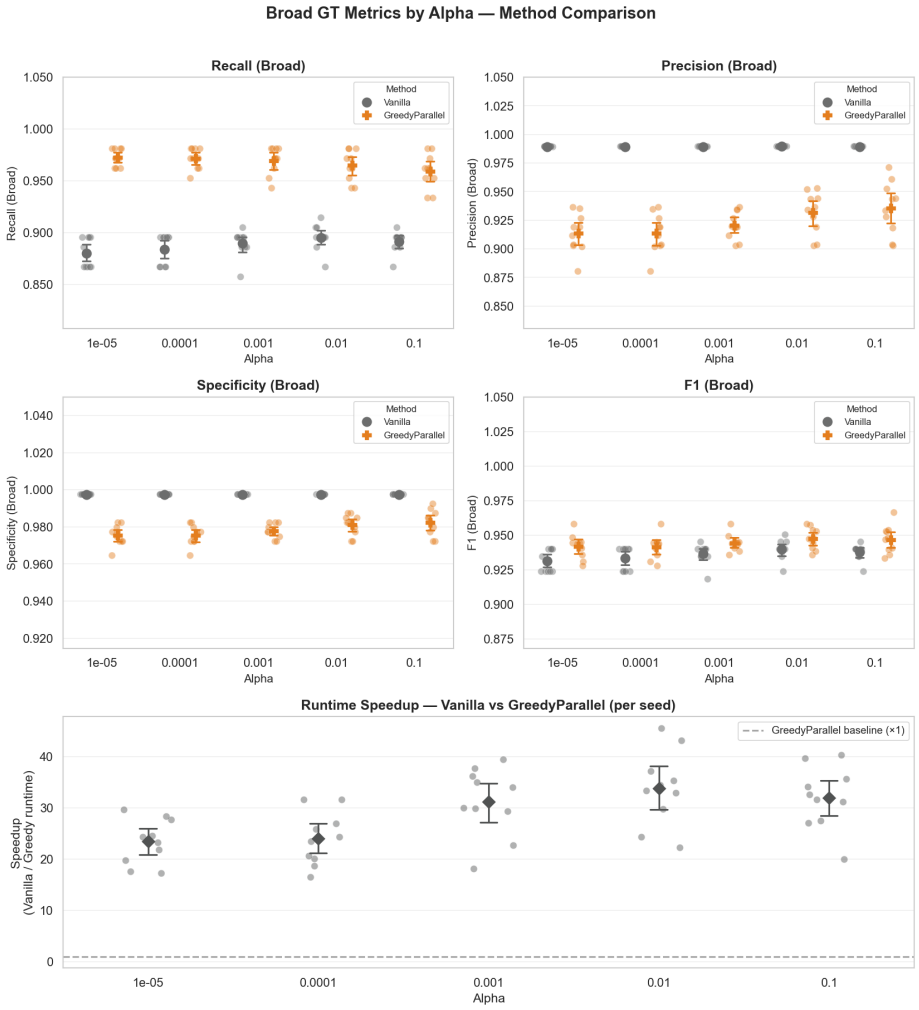


Fig. 1. Final-state ground-truth metrics across significance levels (α) for Vanilla Boruta and Greedy Boruta on the synthetic dataset. Individual runs are shown as semi-transparent dots; markers with caps show mean ± SE.

Figure **??** shows the final-state metric distributions. Greedy Boruta achieves equal or higher Recall than Vanilla Boruta at every α level, in line with Theorem **??**. The cost is a modest reduction in Precision and Specificity: across individual runs, Greedy Boruta incorrectly confirms between 4 and 11 noise features (out of 395), a small fraction of the noise space. Recall is statistically greater for Greedy Boruta (one-sided Welch t-test, $p < 0.01$) and Precision and Specificity are statistically smaller at all α levels, as expected. No statistically significant difference in F1 was detected between the two methods at any α level (two-sided

Welch t-test, $p > 0.01$), consistent with the competing effects of higher recall and lower precision offsetting each other.

Convergence and Early-Stopping Risk Because Vanilla Boruta's stringent confirmation and rejection criteria often require a large number of iterations to resolve the last few features, an experienced practitioner may monitor progress and tune the `max_iter` and early-stopping parameters accordingly.

A more informative comparison between the two algorithms therefore evaluates how results evolve over iterations rather than only at the final state (cf. Figure **??**). To quantify the risk a practitioner faces when stopping anilla Boruta before convergence, we introduce two metrics. The *Undecidedness factor* $U = N_{\text{tentative}}/N_{\text{total}}$ measures the fraction of features that remain unresolved, and the *Potential miss rate* $M = N_{\text{tentative}}/(N_{\text{tentative}} + N_{\text{confirmed}})$ measures the proportion of potentially relevant features that would be lost if all tentatives were rejected. measures the proportion of potentially relevant features that would be lost if early stopping were triggered and all tentative features were rejected while all of them were actually relevant. Higher values of both metrics indicate greater uncertainty and risk associated with early stopping.

We evaluate these metrics by checkpointing Vanilla Boruta at every iteration and tracking ground-truth metrics, using $\alpha = 0.01$.

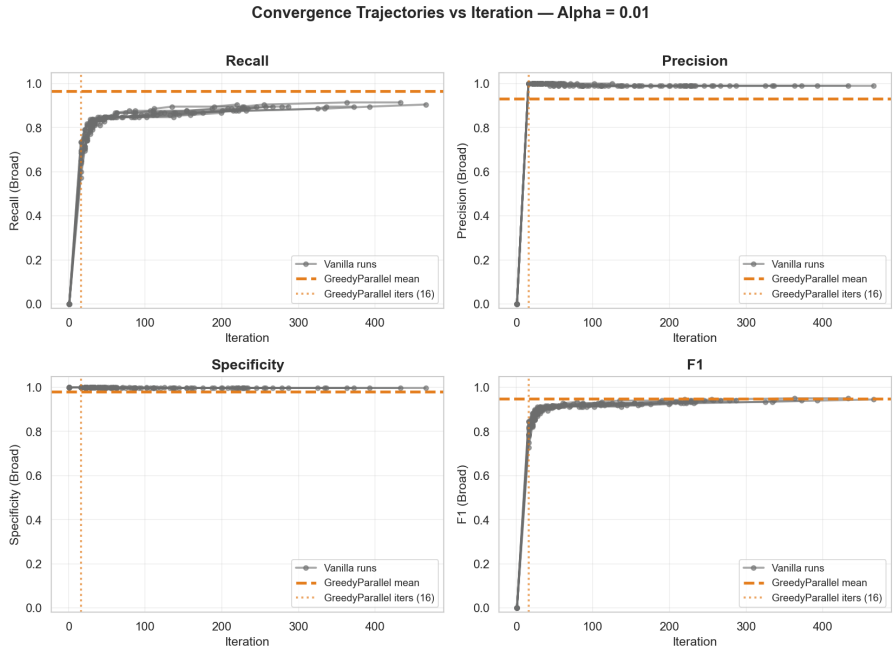


Fig. 2. Per-run convergence trajectories for $\alpha = 0.01$ vs. iteration for all four GT metrics. Each grey line is one seed; the horizontal dashed line represents the Greedy Boruta mean value; The vertical dotted line marks the Greedy Boruta iteration count ($K = 16$).

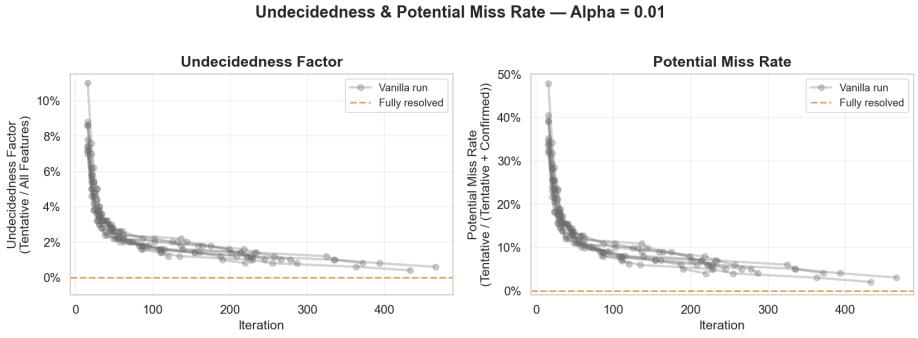


Fig. 3. Undecidedness factor (left) and Potential miss rate (right) for Vanilla Boruta at $\alpha = 0.01$ as a function of iteration number. The vertical dotted line marks the Greedy Boruta iteration count ($K = 16$).

Figure **??** illustrates the computational cost of determining the final features in Vanilla Boruta. The convergence curves reveal a sharp increase in recall during the first tenth of algorithm execution, followed by a plateau-like region where recall improves slowly until either the early stopping criterion is triggered or the maximum number of iterations is reached. Greedy Boruta resolves this inefficiency by confirming all features within K iterations, achieving comparable recall in substantially less time.

One might consider reducing this problem in Vanilla Boruta by lowering the maximum iteration count or tightening early stopping parameters. However, as Figure **??** shows, this approach carries significant risk: prematurely terminating leaves tentative features to be resolved entirely by the rough fix heuristic. Even after 100 iterations—over six times the Greedy Boruta iteration count—approximately 2% of the initial feature set remains unresolved, and the potential miss rate is ∼10%, indicating that early termination would risk discarding over 10% of features that might eventually be confirmed as relevant.

Runtime Speedup Figure **??** also displays per-seed speedup ratios, computed as the ratio of Vanilla Boruta runtime to Greedy Boruta runtime for each random seed. Greedy Boruta achieves consistent speedups of 20–45× across all five α values, with mean speedups ranging from approximately 22× at $\alpha = 10^{-5}$ to 34× at $\alpha = 10^{-2}$. As noted in Section **??**, this speedup combines the effect of fewer iterations (the dominant factor) with outer iteration-level parallelism (a secondary factor). The speedup does not vary monotonically with α due to competing effects in the Vanilla Boruta algorithm: at higher α values, the relaxed statistical testing threshold allows Vanilla Boruta to converge more readily and terminate sooner, whereas at smaller α values, the stringent confirmation and rejection criteria are so difficult to satisfy that early stopping triggers more frequently, also shortening execution.

Combined with the quality results presented in Section **??**, Greedy Boruta delivers a 20–45× runtime reduction while achieving equal or superior Recall and comparable F1 scores, establishing it as the preferable choice on this benchmark (TODO: is this last statement "too bold" for a scientific paper?).

4.3 Real-World Benchmarks

To validate Greedy Boruta beyond controlled synthetic settings, we evaluate on six publicly available high-dimensional benchmarks spanning classification and regression tasks (Table **??**). Because ground-truth relevant feature sets are not available for these datasets, we assess selection quality indirectly: both methods select features independently, a downstream model (Light Gradient Boosting Method [**?**]) is trained on the selected subset, and performance is measured on using cross-validation (5 folds) and weighted F1 (classification) or R^2 (regression). Each (dataset, α, method) configuration is repeated over $R = 10$ independent random seeds; results are reported as mean ± SE.

Table 3. Real-world benchmark datasets.

Dataset	Domain	n	p	Task	Source
Madelon	Artificial	2,000	500	Classification	[**?**]
Gisette	Digit recog.	6,000	5,000	Classification	[**?**]
Arcene	Mass spectrom.	100	10,000	Classification	[**?**]
Dexter	Text classif.	300	20,000	Classification	[**?**]
Isolet	Speech recog.	7,797	617	Classification	[**?**]
Riboflavin	Gene expression	71	4,088	Regression	[**?**]

The significance level is swept over $\alpha \in \{10^{-1}, 10^{-3}, 10^{-5}\}$, a reduced set, since these experiments are much more computationally intensive. All other experimental settings (base estimator, parallelism configuration, hardware) are identical to those described in Section **??**. Vanilla Boruta is compared using the *strict* feature set, i.e., only features confirmed by the binomial test or the rough fix heuristic, excluding tentative features (TODO: is this true? Verify).

Downstream Performance Table **??** presents the downstream performance, runtime, and speedup for each dataset and significance level. To assess whether the observed performance differences are statistically meaningful, we conduct one-sided Welch t-tests at significance level 0.01: one testing whether Greedy Boruta is significantly better, and the converse. The "Sig." column reports the outcome: $\approx$ indicates no statistically significant difference (tie), $\blacktriangleright$ indicates Vanilla is significantly better, and $\blacktriangleleft$ indicates Greedy is significantly better.

Table 4. Real-world benchmark results (mean ± SE over $R = 10$ seeds). *Score* is test-set F1 (classification) or R^2 (regression). *Sig.*: directional one-sided Welch t-test at level 0.01; ▶ = Vanilla significantly better, ◀ = Greedy significantly better, ≈ = no significant difference. *Speedup* = Vanilla time / Greedy time.

		Greedy Boruta		Vanilla Boruta			
Dataset	α	Score	Time (s)	Score	Time (s)	Sig.	Speedup
Madelon	10^{-1}	$.856 \pm .002$	2.4	$.862 \pm .002$	34.7	▶	14.6×
	10^{-3}	$.856 \pm .002$	3.2	$.866 \pm .001$	34.8	▶	10.9×
	10^{-5}	$.855 \pm .002$	4.1	$.866 \pm .001$	39.1	▶	9.5×
Arcene	10^{-1}	$.868 \pm .005$	11.2	$.878 \pm .004$	184.3	≈	16.4×
	10^{-3}	$.875 \pm .005$	14.0	$.879 \pm .003$	203.3	≈	14.6×
	10^{-5}	$.875 \pm .006$	19.9	$.878 \pm .005$	215.7	≈	10.9×
Gisette	10^{-1}	$.982 \pm .0003$	35.4	$.982 \pm .0003$	814.0	≈	23.0×
	10^{-3}	$.982 \pm .0002$	49.8	$.983 \pm .0001$	801.3	≈	16.1×
	10^{-5}	$.982 \pm .0002$	61.5	$.982 \pm .0002$	824.1	≈	13.4×
Dexter	10^{-1}	$.935 \pm .002$	21.0	$.934 \pm .002$	165.6	≈	7.9×
	10^{-3}	$.936 \pm .002$	29.2	$.936 \pm .002$	185.6	≈	6.4×
	10^{-5}	$.937 \pm .002$	34.1	$.936 \pm .002$	212.9	≈	6.2×
Riboflavin	10^{-1}	$.581 \pm .019$	47.6	$.539 \pm .020$	100.6	≈	2.1×
	10^{-3}	$.584 \pm .017$	64.6	$.542 \pm .021$	125.9	≈	1.9×
	10^{-5}	$.583 \pm .017$	86.8	$.545 \pm .021$	154.5	≈	1.8×
Isolet	10^{-1}	$.959 \pm .0003$	5.1	$.959 \pm .0002$	340.2	≈	66.2×
	10^{-3}	$.959 \pm .0003$	7.3	$.959 \pm .0003$	355.0	≈	48.8×
	10^{-5}	$.959 \pm .0003$	9.3	$.958 \pm .0002$	393.6	≈	42.5×

Of the 18 (dataset, α) configurations, 15 show no statistically significant difference in downstream performance between the two methods. In 3 configurations (Madelon at all three α levels), Vanilla Boruta achieves a statistically significant but practically small advantage: the largest observed gap is ∼0.01 in F1 (Madelon, $\alpha = 10^{-5}$).

Importantly, these performance differences, whether favourable or unfavourable to Greedy Boruta, are negligible in absolute terms, while the runtime reductions are substantial. Speedups range from 2.2× (Riboflavin, $\alpha = 10^{-5}$) to 66.2× (Isolet, $\alpha = 0.1$), with larger speedups observed on datasets where Vanilla Boruta requires more iterations to resolve features, which depends on the signal-to-noise characteristics of the data rather than dimensionality alone. (TODO: should we include this comment? Is this true?).

Figure **??** shows the full score distributions across seeds, confirming that the inter-method differences are small relative to the run-to-run variability within each method.

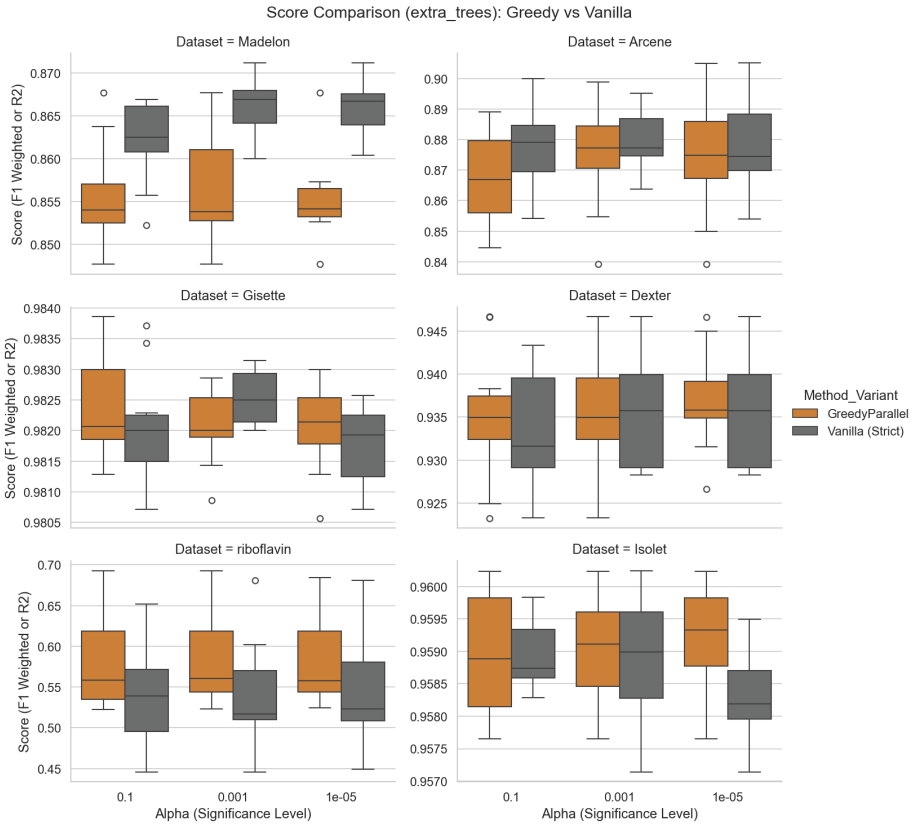


Fig. 4. Downstream performance (weighted F1 or R^2) across significance levels for each dataset (5 seeds per configuration). The narrow interquartile ranges and overlapping boxes confirm that performance differences between methods are small relative to run-to-run variability.

Pareto Efficiency: Performance vs. Cost To visualise the performance–cost trade-off across all datasets simultaneously, Figure **??** plots the mean downstream score against mean wall-clock time for each (dataset, α, method) configuration. Points sharing the same colour represent the same dataset; circles denote Greedy Boruta and crosses denote Vanilla Boruta; within each (dataset, method) group, the different points correspond to different α values.

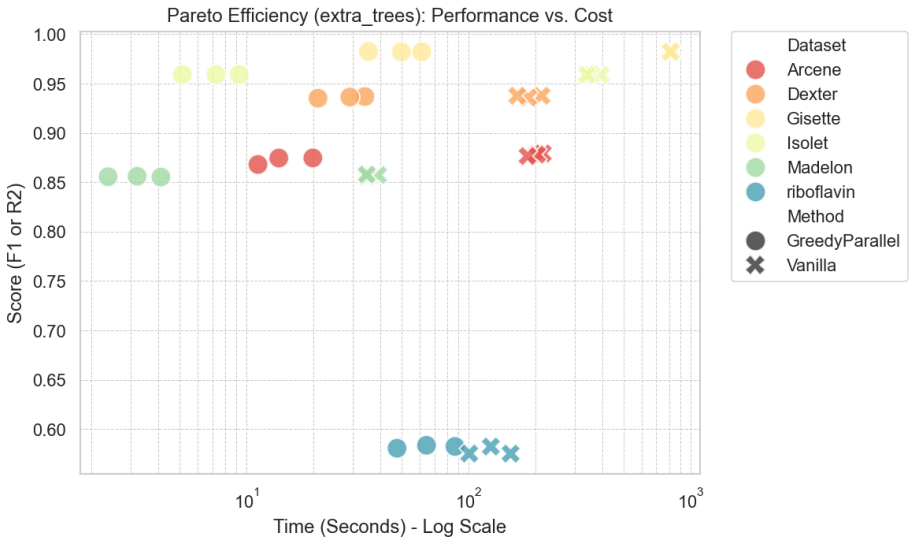


Fig. 5. Pareto efficiency plot: downstream score vs. wall-clock time (log scale) for all (dataset, α, method) configurations. Each colour represents a dataset; circles are Greedy Boruta, crosses are Vanilla Boruta. Points of the same colour and marker correspond to different α levels. Greedy Boruta consistently occupies the upper-left region (high performance, low cost), Pareto-dominating or matching Vanilla Boruta on every dataset.

Across all six datasets, the Greedy Boruta points (circles) sit at or near the same vertical position as the corresponding Vanilla Boruta points (crosses) but are displaced one to two orders of magnitude to the left on the time axis. This means Greedy Boruta achieves equivalent downstream performance at a fraction of the computational cost, Pareto-dominating Vanilla Boruta in the majority of configurations.

For the practitioner, this plot encapsulates the central practical claim of this paper: Greedy Boruta offers an almost strictly better trade-off between selection quality and computational cost across a wide range of dataset characteristics and significance levels.

Convergence on Real Data We repeat the convergence analysis from Section **??** on real data by checkpointing Vanilla Boruta at every iteration on Gisette ($p = 5{,}000$) with $\alpha = 0.1$.

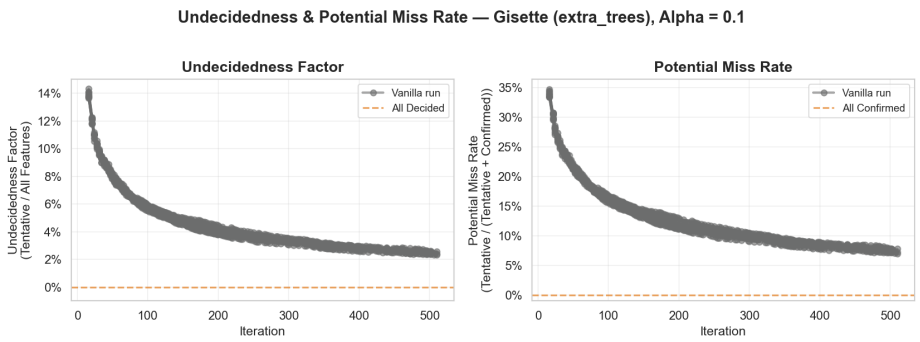


Fig. 6. Left: Undecidedness factor for Vanilla Boruta on Gisette with $\alpha = \{10^{-1}\}$ as a function of iteration number . Right: Potential miss rate for Vanilla Boruta on Gisette with $\alpha = \{10^{-1}\}$, as a function of iteration number. 10 seeds per panel. The dashed lines marks complete resolution (0%).

Figure **??** shows that the undecidedness factor follows the same characteristic decay observed on synthetic data: a rapid initial drop as clearly informative and clearly irrelevant features are resolved, followed by a long tail where the remaining borderline features resist classification. After the full 512-iteration budget, 2–3% of Gisette's 5,000 features remain tentative across all α levels and seeds—corresponding to 100–200 features whose status is ultimately resolved by the rough fix heuristic rather than the principled binomial test.

Figure **??** translates this into practical risk: the potential miss rate, which measures the fraction of possibly relevant features that would be lost if tentatives were discarded, plateaus at approximately 6% with $\alpha = 0.1$, which is a lenient configuration for Vanilla Boruta. On this real high-dimensional dataset, a practitioner stopping Vanilla Boruta early faces uncertainty about whether the selected feature set is complete. Greedy Boruta eliminates this uncertainty entirely: on Gisette with $\alpha = 0.1$, convergence requires $K = \lceil \log_2(5000/0.1) \rceil = 16$ iterations, resolving all 5,000 features with no tentatives and no reliance on heuristic post-processing.

Summary The real-world benchmarks reinforce the conclusions from the synthetic experiments. Greedy Boruta delivers speedups of 2×–66× across six datasets spanning $p = 500$ to $p = 20{,}000$ features, with downstream model performance that is statistically indistinguishable from Vanilla Boruta in 15 of 18 configurations and practically equivalent in the remaining cases (maximum gap ∼0.01 in F1 or R^2). The convergence analysis on Gisette shows that the tentative-feature problem persists on real data with practical consequences. Together with the Pareto efficiency analysis, these results establish Greedy Boruta as the preferred method when the all-relevant paradigm is appropriate and downstream filtering is available.

5 Discussion

5.1 The FPR–Convergence Trade-off in Practice

The theoretical analysis in Section **??** establishes that Greedy Boruta exchanges Vanilla Boruta's near-zero false positive rate for a bounded FPR of $K/(p+1)$, where $K = \lceil \log_2(p/\alpha) \rceil$. The empirical results allow us to assess whether this trade-off is acceptable in practice.

On the synthetic benchmark, Greedy Boruta incorrectly confirms between 4 and 11 noise features out of 395 across all seeds and α levels, a small fraction of the noise space that produces no statistically significant degradation in F1 (Section **??**). On the six real-world benchmarks, downstream performance is statistically indistinguishable between the two methods in 15 of 18 configurations, with the remaining 3 (all on Madelon) showing a gap of at most $\sim$0.01 in F1 (Table **??**). Madelon has the smallest feature count in our benchmark suite ($p = 500$), which is precisely the regime where the FPR bound from Theorem **??** is least favorable. At larger p, the bound tightens: for $p = 5{,}000$ (Gisette) or $p = 20{,}000$ (Dexter), the per-feature FPR falls below 1%, and no significant performance difference is observed. This self-correcting behavior aligns the method's strongest statistical guarantees with its strongest computational advantages.

From a broader statistical perspective, Greedy Boruta can be understood as adopting a power-maximizing philosophy within the all-relevant framework. It sacrifices Type I error control (from near-zero to bounded) in exchange for maximum recall, mirroring the well-known tension between family-wise error rate control and false discovery rate control in the multiple testing literature. The justification is specific to the all-relevant paradigm: false negatives represent permanently lost information, while false positives can be filtered by downstream regularization, cross-validation, or minimal-optimal selection methods.

5.2 Practical Guidance

Greedy Boruta is best suited to high-dimensional settings ($p \geq 500$), where the FPR bound remains below 5% across all α values evaluated (Table **??**) and the computational savings are largest (up to 66$\times$ on Isolet). It is also well suited to automated or production pipelines, where the deterministic termination in $K = \lceil \log_2(p/\alpha) \rceil$ iterations eliminates the need to tune `max_iter`, `early_stopping`, and `n_iter_no_change`. As a consequence, Greedy Boruta produces a complete partition of features into confirmed and rejected sets with no tentative features, removing the dependence on the rough fix heuristic that Vanilla Boruta uses to resolve remaining tentatives at termination.

Vanilla Boruta may remain preferable in low-dimensional problems ($p < 100$), where the FPR bound can exceed 10% and the computational cost of the standard algorithm is more likely manageable. It is also the safer choice when false positives are extremely costly and no downstream filtering mechanism is available.

5.3 Limitations

The FPR analysis relies on the shadow exchangeability assumption (Assumption **??**), which requires that irrelevant features and shadow features be identically distributed with respect to the importance measure. Violations of this assumption, for instance due to importance measure biases toward high-cardinality features, affect both methods. However, features with marginal biases that produce occasional but non-significant hits are confirmed by Greedy Boruta but may remain tentative or be rejected under Vanilla Boruta's stricter criterion.

The parallelization benefit described in Section **??** depends on the available hardware relative to the base estimator's internal parallelism. On typical workstations, the tree-building parallelism of the base estimator (e.g., `n_jobs=-1` in `ExtraTreesClassifier`) is sufficient to saturate all available CPU cores during a single iteration, leaving little room for outer iteration-level parallelism to contribute. In large computing clusters, where, for example, the number of available cores exceeds the number of trees being built per iteration, the outer parallelism can provide a more substantial additional speedup. In both regimes, the dominant source of speedup remains the reduction in iteration counts.

Finally, Greedy Boruta inherits all base estimator limitations from Vanilla Boruta, including sensitivity to the choice of importance measure and the assumption that tree-based ensembles adequately capture feature relevance.

6 Conclusion

We have presented Greedy Boruta, a modification to the Boruta algorithm that replaces the binomial test confirmation criterion with a single-hit rule: a feature is confirmed if it ever exceeds the maximum shadow importance in any iteration. This change yields three formal guarantees: recall at least as high as Vanilla Boruta (Theorem **??**), convergence in exactly $K = \lceil \log_2(p/\alpha) \rceil$ iterations under Bonferroni correction (Theorem **??**), and a per-feature false positive rate bounded above by $K/(p+1)$ (Theorem **??**). A further consequence is that FDR correction becomes trivially redundant (Theorem **??**), collapsing the multiple testing correction landscape to a single Boolean parameter. Experiments on a synthetic benchmark with known ground truth and six real-world high-dimensional datasets confirm these theoretical properties and demonstrate substantial speedups with negligible impact on downstream model performance.

We recommend Greedy Boruta as the default choice for all-relevant feature selection in pipelines (TODO: Too bold once again?) where downstream filtering is available, particularly in high-dimensional settings where its FPR bound is tightest and its computational advantage is largest. An implementation is publicly available as `GreedyBorutaPy`. Future work includes extending the greedy criterion to alternative importance measures such as SHAP and permutation importance, and investigating adaptive significance level selection strategies.

References

1. Kursa, M.B., Rudnicki, W.R.: Feature selection with the Boruta package. Journal of Statistical Software **36**(11), 1–13 (2010)

WRITE: Add all necessary references:

- Boruta original paper
- BorutaPy implementation
- Feature selection surveys (Guyon & Elisseeff, etc.)
- mRMR, Lasso, RFE papers
- Extra Trees, Random Forest, XGBoost, LightGBM
- Dataset sources (UCI, OpenML)
- Multiple testing literature
- Any other relevant citations

A Proofs

A.1 Proof of Theorem ?? (Recall Guarantee)

Proof. Vanilla Boruta confirms features through two mechanisms: the binomial test during execution and, upon termination, the rough fix heuristic for remaining tentative features. We show that both require $h_f \geq 1$ hits.

Case 1: Binomial test confirmation. Consider any feature f confirmed by the binomial test. This requires $P(X \geq h_f \mid n = K, p = 0.5) < \alpha$, where $X \sim \text{Binomial}(K, 0.5)$. For $h_f = 0$, this probability equals $P(X \geq 0) = 1 > \alpha$ for any $\alpha < 1$, so the test cannot reject. Therefore $h_f \geq 1$.

Case 2: Rough fix confirmation. The rough fix [**?**] confirms a tentative feature f if its median importance exceeds the median of the per-iteration maximum shadow importances:

$$\text{median}(f_1, \ldots, f_K) > \text{median}(M_1, \ldots, M_K),$$

where f_i is the importance of feature f in iteration i and $M_i = \max_j S_j^{(i)}$ is the maximum shadow importance.

If $h_f = 0$, then $f_i \leq M_i$ for every iteration i. For any threshold $t \in \mathbb{R}$, if $M_i \leq t$ then $f_i \leq M_i \leq t$, so

$$|\{i : f_i \leq t\}| \geq |\{i : M_i \leq t\}|$$

for all t. The empirical distribution of the f_i is therefore stochastically dominated by that of the M_i. Stochastic dominance preserves quantile ordering, so $\text{median}(f) \leq \text{median}(M)$, and the rough fix cannot confirm a feature with zero hits.

Conclusion. In both cases, confirmation by Vanilla Boruta requires $h_f \geq 1$. By the Greedy Boruta confirmation criterion (Algorithm **??**, line 14), any such feature is confirmed, so $f \in \mathcal{C}_G$. Since this holds for all $f \in \mathcal{C}_V$, we conclude $\mathcal{C}_V \subseteq \mathcal{C}_G$.

A.2 Proof of Theorem ?? (Convergence Guarantee)

Proof. At iteration K, partition all features into $\mathcal{H}^+ = \{j : \text{hits}[j] \geq 1\}$ and $\mathcal{H}^0 = \{j : \text{hits}[j] = 0\}$.

By Algorithm **??** (line 14), all features in $\mathcal{H}^+$ are confirmed.

For any $j \in \mathcal{H}^0$, the rejection test evaluates $H_0 : p \geq 0.5$ vs. $H_1 : p < 0.5$, where p is the probability that feature j beats the maximum shadow importance. The p-value is:

$$\text{p-value} = P(X \leq 0 \mid n = K, p = 0.5) = (0.5)^K = \frac{1}{2^K}$$

All features in $\mathcal{H}^0$ have this identical p-value. Under Bonferroni correction across $|\mathcal{H}^0| \leq p$ features, rejection requires $(0.5)^K < \alpha/p$, i.e., $2^K > p/\alpha$, i.e., $K > \log_2(p/\alpha)$. The minimum integer satisfying this is $K = \lceil \log_2(p/\alpha) \rceil$.

At this iteration all features in $\mathcal{H}^0$ are rejected and all in $\mathcal{H}^+$ are confirmed, so the algorithm terminates with no tentative features.

Iterations without Bonferroni correction. Without Bonferroni correction, rejection requires $(0.5)^K < \alpha$, giving $K = \lceil \log_2(1/\alpha) \rceil$. Table **??** shows values for common α.

Table 5. Number of iterations $K = \lceil \log_2(1/\alpha) \rceil$ required for convergence without Bonferroni correction.

α	K
0.100	4
0.050	5
0.010	7
0.001	10
10^{-4}	14
10^{-5}	17

A.3 Proof of Theorem ?? (FDR Correction Triviality)

Proof. From the proof of Theorem **??**, any tentative feature j at iteration K has $\text{hits}[j] = 0$ and p-value $p^* = 1/2^K$.

Let $\mathcal{T} = \{j : \text{hits}[j] = 0\}$ with $|\mathcal{T}| = m$. All features in $\mathcal{T}$ share the same p-value p^*.

The Benjamini-Hochberg procedure orders p-values $p_{(1)} \leq \cdots \leq p_{(m)}$ and finds the largest i such that $p_{(i)} \leq (i/m) \cdot \alpha$. Since all p-values are identical ($p_{(i)} = p^*$ for all i), this reduces to checking whether $p^* \leq \alpha$. From the construction of $K = \lceil \log_2(1/\alpha) \rceil$, we have $p^* = 1/2^K < \alpha$, so the condition holds at $i = m$ and all tentative features are rejected. This outcome is identical to the uncorrected test, making FDR correction trivial.

A.4 Proof of Theorem ?? (FPR of Greedy Boruta)

Proof. Under Assumption **??**, the importance of an irrelevant feature and the p shadow importances form $p + 1$ exchangeable random variables. By symmetry, each is equally likely to be the maximum, so the per-iteration hit probability is $\theta_0 = 1/(p + 1)$.

Let H denote the hit count for an irrelevant feature across K independent iterations. Then $H \sim \text{Binomial}(K, \theta_0)$, and:

$$P(\text{false positive}) = P(H \geq 1) = 1 - P(H = 0) = 1 - (1 - \theta_0)^K = 1 - \left(\frac{p}{p+1}\right)^K.$$

For the upper bound, Bernoulli's inequality gives $(1 - x)^K \geq 1 - Kx$ for $x \in [0, 1]$ and integer $K \geq 1$. Setting $x = 1/(p + 1)$:

$$\left(\frac{p}{p+1}\right)^K \geq 1 - \frac{K}{p+1},$$

so $P(\text{false positive}) \leq K/(p + 1)$.

Comparison with Vanilla Boruta. Under the same exchangeability assumption, Vanilla Boruta's FPR is negligible. To be confirmed after K iterations, a feature must accumulate at least h^* hits where $P(\text{Binomial}(K, 0.5) \geq h^*) \leq \alpha$. For a feature with true hit rate $\theta_0 = 1/(p + 1)$, the probability of reaching h^* hits is $P(\text{Binomial}(K, \theta_0) \geq h^*)$, which is vanishingly small. For example, with $\alpha = 0.001$ and $p = 500$: Greedy Boruta runs $K = 19$ iterations with $h^* = 17$ required for vanilla confirmation, giving $P(\text{FP} \mid \text{Vanilla}) \approx 10^{-44}$ compared to $P(\text{FP} \mid \text{Greedy}) \approx 3.73\%$. This gap reflects the fundamental trade-off: Greedy Boruta exchanges a near-zero FPR for guaranteed convergence and computational efficiency.